



CÓDIGO BASE

JAVA Y PROGRAMACIÓN ORIENTADA A OBJETOS

Clases, herencia y polimorfismo explicados con ejemplos ejecutables

TINTA DIGITAL

Nivel: Intermedio

Java y Programación Orientada a Objetos

Clases, herencia y polimorfismo explicados con ejemplos ejecutables

Colección: Código Base

Nivel: Nivel: Intermedio

Editorial: Tinta Digital

Edición: 1.^a edición digital, 2026

Este ebook forma parte del catálogo autogenerado de Tinta Digital. Su contenido puede reproducirse con fines de estudio personal citando la fuente.

Índice

- 01 — Clases y objetos
- 02 — Encapsulación
- 03 — Herencia
- 04 — Polimorfismo
- 05 — Abstracción: clases abstractas e interfaces
- 06 — Colecciones esenciales
- 07 — Manejo de excepciones

01. Clases y objetos

“Una clase es un plano; un objeto es la casa construida a partir de ese plano.”

En Java, todo gira en torno a clases: plantillas que definen atributos (datos) y métodos (comportamiento). Un objeto es una instancia concreta de una clase.

```
public class Coche {  
    private String modelo;  
    private int velocidad;  
  
    public Coche(String modelo) {  
        this.modelo = modelo;  
        this.velocidad = 0;  
    }  
  
    public void acelerar(int incremento) {  
        this.velocidad += incremento;  
    }  
  
    public String getModelo() {  
        return modelo;  
    }  
}
```

El constructor `Coche(String modelo)` se ejecuta al crear el objeto con `new Coche("Civic")`. Los métodos `get` exponen los datos internos de forma controlada, en lugar de dejar los atributos accesibles directamente.

02. Encapsulación

Encapsular significa ocultar el estado interno de un objeto y exponer solo lo necesario a través de métodos públicos. Java usa modificadores de acceso para controlarlo.

- **private:** solo visible dentro de la misma clase.
- **protected:** visible en la clase, el paquete y las subclases.
- **public:** visible desde cualquier parte del programa.
- **(sin modificador):** visible solo dentro del mismo paquete.

La regla práctica: los atributos casi siempre deben ser private, y el acceso se gestiona mediante métodos getter y setter que pueden incluir validación.

```
public void setVelocidad(int velocidad) {  
    if (velocidad >= 0) {  
        this.velocidad = velocidad;  
    }  
}
```

03. Herencia

La herencia permite que una clase (subclase) reutilice y extienda el comportamiento de otra (superclase) mediante `extends`.

```
public class CocheElectrico extends Coche {  
    private int autonomiaKm;  
  
    public CocheElectrico(String modelo, int autonomiaKm) {  
        super(modelo);  
        this.autonomiaKm = autonomiaKm;  
    }  
  
    public int getAutonomiaKm() {  
        return autonomiaKm;  
    }  
}
```

`super(modelo)` invoca el constructor de la clase padre. `CocheElectrico` hereda automáticamente el método `acelerar()` sin necesidad de reescribirlo.

Cuándo usar herencia

La herencia modela una relación "es-un": un coche eléctrico es un coche. Cuando la relación es más bien "tiene-un" (un coche tiene un motor), suele ser preferible la composición: incluir un objeto `Motor` como atributo en lugar de heredar de él.

04. Polimorfismo

Polimorfismo significa que un mismo método puede comportarse de forma distinta según el objeto que lo ejecute.

```
public class Coche {  
    public String tipoMotor() {  
        return "combustión";  
    }  
}  
  
public class CocheElectrico extends Coche {  
    @Override  
    public String tipoMotor() {  
        return "eléctrico";  
    }  
}  
  
Coche c = new CocheElectrico("Model 3", 400);  
System.out.println(c.tipoMotor()); // "eléctrico"
```

Aunque la variable `c` está declarada como `Coche`, Java ejecuta en tiempo de ejecución el método sobrescrito en `CocheElectrico`. Esto permite escribir código genérico que funciona con cualquier subclase, sin conocer su tipo exacto de antemano.

05. Abstracción: clases abstractas e interfaces

Una clase abstracta no puede instanciarse directamente; define un esqueleto común para sus subclases.

```
public abstract class Figura {
    public abstract double area();

    public void describir() {
        System.out.println("Área: " + area());
    }
}

public class Circulo extends Figura {
    private double radio;
    public Circulo(double radio) { this.radio = radio; }

    @Override
    public double area() {
        return Math.PI * radio * radio;
    }
}
```

Interfaces

Una interfaz define un contrato de métodos que una clase se compromete a implementar, sin imponer una jerarquía de herencia. Una clase puede implementar varias interfaces a la vez, algo que Java no permite con herencia de clases.

```
public interface Comparable {
    int compareTo(T otro);
}

public class Producto implements Comparable {
    private double precio;
    public int compareTo(Producto otro) {
        return Double.compare(this.precio, otro.precio);
    }
}
```


06. Colecciones esenciales

El paquete `java.util` ofrece estructuras de datos listas para usar.

- **ArrayList**: lista ordenada de tamaño dinámico, acceso rápido por índice.
- **HashMap**: pares clave-valor, búsqueda muy rápida por clave.
- **HashSet**: colección sin elementos duplicados, sin orden garantizado.
- **LinkedList**: lista enlazada, eficiente en inserciones/eliminaciones frecuentes.

```
List nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");

Map edades = new HashMap<>();
edades.put("Ana", 22);

for (String nombre : nombres) {
    System.out.println(nombre + ": " + edades.get(nombre));
}
```

07. Manejo de excepciones

Java gestiona errores en tiempo de ejecución mediante excepciones, capturadas con bloques try-catch.

```
try {  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
} finally {  
    System.out.println("Bloque finally: se ejecuta siempre");  
}
```

Las excepciones comprobadas (checked), como `IOException`, obligan a declararlas o capturarlas explícitamente. Las no comprobadas (unchecked), como `NullPointerException`, no lo exigen, pero conviene prevenirlas con validaciones.